

Gateway LoRaWAN RAK7371 + Supabase no Armbian

Manual de Instalação e Configuração

Stack: ChirpStack • Docker • MQTT • Python Bridge
• Supabase

Região: AU915 (Brasil/ANATEL)

Wilson Cosmo
github.com/WCosmo

7 de setembro de 2026

Sumário

1	Introdução	2
1.1	Pré-requisitos	2
1.2	Repositório Oficial	2
2	Preparação do Sistema	2
2.1	Atualização do Armbian	2
2.2	Instalação do Docker e Docker Compose	3
2.2.1	Configuração de grupos de usuário	3
2.3	Instalação das dependências Python	3
3	Clone e Configuração do Projeto	4
3.1	Clone do repositório	4
3.2	Configuração do ambiente (.env)	4
3.3	Setup interativo (alternativa recomendada)	5
4	Configuração do Banco de Dados no Supabase	5
5	Inicialização dos Serviços	6
5.1	ChirpStack (Network Server)	6
5.2	Packet Forwarder (RAK7371 via USB)	6
5.3	Bridge Python (MQTT → Supabase)	6
5.3.1	Execução como serviço systemd (recomendado)	7
6	Configuração no ChirpStack	7
6.1	Cadastro do Gateway	7
6.2	Cadastro do Dispositivo (Sensor)	8
7	Comandos Úteis	8
8	Solução de Problemas	9
8.1	Gateway aparece “Offline”	9
8.2	Erro sx1261 nos logs do packet forwarder	9
8.3	Dados não chegam no Supabase	9
8.4	Payload chega em base64	9
8.5	Problemas específicos do Armbian	9
8.5.1	RAK7371 não aparece no lsusb	9
8.5.2	Erro de permissão USB	9
8.5.3	Containers reiniciam em loop	10
9	Considerações de Segurança	10
10	Conclusão	10

1 Introdução

Este manual descreve o passo a passo completo para montar um **gateway LoRaWAN** utilizando o concentrador **RAK7371** em uma placa ARM rodando **Armbian**, com integração automática dos dados no **Supabase** via **ChirpStack**.

A arquitetura completa do projeto é ilustrada na Figura 1.

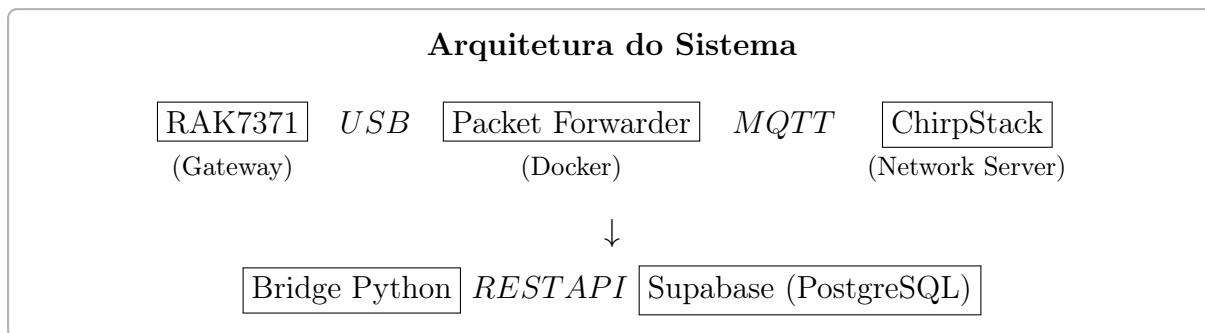


Figura 1: Fluxo de dados do gateway LoRaWAN

1.1 Pré-requisitos

- Placa ARM com **Armbian** (Debian/Ubuntu flavor)
- **RAK7371** conectado via USB
- Acesso à internet
- Acesso root ou `sudo`
- Conta no **Supabase** (<https://supabase.com>)

1.2 Repositório Oficial

<https://github.com/WCosmo/lorawan-rak7371-supabase>

2 Preparação do Sistema

2.1 Atualização do Armbian

Antes de iniciar, atualize todos os pacotes do sistema:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl wget htop udev
```

2.2 Instalação do Docker e Docker Compose

No Armbian (ARM64/ARMHF), o Docker é instalado via repositório oficial. Execute:

```
curl -fsSL https://get.docker.com | sh
```

⚠ Atenção

Em algumas imagens mínimas do Armbian, o comando `newgrp` pode não estar disponível. Caso ocorra `command not found`, utilize a alternativa descrita na Seção 2.2.1.

2.2.1 Configuração de grupos de usuário

Adicione o usuário ao grupo `docker` para evitar o uso de `sudo` em todos os comandos:

```
sudo usermod -aG docker $USER
```

Para recarregar os grupos na sessão atual, utilize **uma** das alternativas abaixo:

Opção 1 (recomendada): Reinicie a sessão
`exit` e faça login novamente.

Opção 2: Use `su`
`su - $USER`

Opção 3: Continue com `sudo`
`sudo docker compose up -d`

💡 Dica

Se o comando `newgrp docker` retornar `command not found`, instale o pacote `login` com `sudo apt install -y login` ou simplesmente use `su - $USER`.

2.3 Instalação das dependências Python

O bridge Python requer ambiente virtual. Instale os pacotes necessários:

```
sudo apt install -y docker.io docker-compose-plugin \
python3 python3-pip python3-venv
```

 Dica

Se a placa possui menos de **2 GB de RAM**, configure **swap** antes de subir os containers:

```
sudo falldate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

3 Clone e Configuração do Projeto

3.1 Clone do repositório

```
cd ~
git clone https://github.com/WCosmo/lorawan-rak7371-supabase.git
cd lorawan-rak7371-supabase
```

A estrutura do projeto é a seguinte:

```
lorawan-rak7371-supabase/
|-- docker-compose.yml          # ChirpStack + PostgreSQL + Redis + Mosquitto
|-- .env.example                # Template de variáveis de ambiente
|-- setup.sh                    # Script de setup interativo
|-- packet-forwarder/
|   |-- docker-compose.yml      # Packet Forwarder do RAK7371
|-- chirpstack/
|   |-- configuration/chirpstack/chirpstack.toml # AU915 (Brasil)
|-- bridge/
|   |-- bridge.py                # Script Python: MQTT -> Supabase
|   |-- requirements.txt
|-- supabase_schema.sql         # Schema da tabela no Supabase
```

3.2 Configuração do ambiente (.env)

O projeto inclui um arquivo `.env.example` que serve de template. Copie-o para `.env`:

```
cp .env.example .env
nano .env
```

Preencha as variáveis obrigatórias conforme a Tabela 1.

Tabela 1: Variáveis de ambiente do .env

Variável	Onde obter
GATEWAY_EUI	Execute: <code>docker compose -f packet-forwarder/docker-compose.yml run -rm udp-packet-forwarder find_concentrator</code>
SUPABASE_URL	Dashboard Supabase → Project Settings → API → URL
SUPABASE_KEY	Dashboard Supabase → API → service_role key
CHIRPSTACK_SECRET	Deixe em branco (gerado automaticamente)

⚠ Atenção

ATENÇÃO: Utilize obrigatoriamente a `service_role` key, **nunca** a `anon` key. A `service_role` key possui privilégios de escrita no banco e **não deve ser exposta em frontend**.

3.3 Setup interativo (alternativa recomendada)

O repositório inclui o script `setup.sh`, que automatiza a detecção do EUI e a geração do secret:

```
chmod +x setup.sh
./setup.sh
```

O script detecta automaticamente o EUI do RAK7371 e configura o ambiente interativamente.

4 Configuração do Banco de Dados no Supabase

No **SQL Editor** do dashboard do Supabase, execute o schema abaixo para criar a tabela de recebimento dos dados:

```
CREATE TABLE lorawan_data (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  device_eui TEXT NOT NULL,
  device_name TEXT,
  device_profile TEXT,
  data JSONB,
  raw_payload TEXT,
  received_at TIMESTAMPTZ,
  rssi INTEGER,
  snr NUMERIC,
  gateway_id TEXT,
  frequency BIGINT,
```

```
data_rate INTEGER,  
location JSONB,  
created_at TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX idx_lorawan_device_eui ON lorawan_data(device_eui);  
CREATE INDEX idx_lorawan_received_at ON  
lorawan_data(received_at);
```

5 Inicialização dos Serviços

5.1 ChirpStack (Network Server)

Suba os containers do ChirpStack, PostgreSQL, Redis e Mosquitto:

```
cd ~/lorawan-rak7371-supabase  
docker compose up -d
```

Aguarde aproximadamente 30–60 segundos (a primeira inicialização em ARM pode ser mais lenta).

Acesse a interface web em: http://<IP_DO_ARMBIAN>:8080

Login padrão: admin / admin

5.2 Packet Forwarder (RAK7371 via USB)

Conecte o RAK7371 via USB e suba o packet forwarder:

```
cd ~/lorawan-rak7371-supabase/packet-forwarder  
docker compose up -d  
cd ..
```

Dica

Verifique se o RAK7371 foi reconhecido pelo sistema:

```
lsusb  
dmesg | grep -i "rak\\|cp210x\\|ftdi\\|usb"
```

Você deve visualizar algo como Future Technology Devices International ou Silicon Labs CP210x.

5.3 Bridge Python (MQTT → Supabase)

O bridge consome as mensagens MQTT do ChirpStack e insere os dados no Supabase. Como não está containerizado no repositório original, execute no host:

```
cd ~/lorawan-rak7371-supabase/bridge  
python3 -m venv venv
```

```
source venv/bin/activate
pip install -r requirements.txt
python bridge.py
```

5.3.1 Execução como serviço systemd (recomendado)

Para manter o bridge rodando permanentemente (mesmo após reboot), crie um serviço:

```
sudo tee /etc/systemd/system/lorawan-bridge.service > /dev/null
<<EOF
[Unit]
Description=LoRaWAN Bridge MQTT -> Supabase
After=network.target

[Service]
Type=simple
User=$USER
WorkingDirectory=/home/$USER/lorawan-rak7371-supabase/bridge
ExecStart=/home/$USER/lorawan-rak7371-supabase/bridge/venv/bin/python
    bridge.py
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
EOF
```

Habilite e inicie o serviço:

```
sudo systemctl daemon-reload
sudo systemctl enable lorawan-bridge
sudo systemctl start lorawan-bridge
```

Verifique o status:

```
sudo systemctl status lorawan-bridge
```

6 Configuração no ChirpStack

6.1 Cadastro do Gateway

1. Acesse **Tenants** → **ChirpStack** → **Gateways** → **Add gateway**
2. **Gateway ID (EUI)**: insira o valor do `GATEWAY_EUI` do `.env`
3. **Name**: RAK7371-Armbian
4. **Gateway Profile**: selecione o perfil da região **AU915**
5. Salve — o status deve ficar **“Online”**

⚠ Atenção

O projeto está pré-configurado para a região **AU915** (Brasil/ANATEL). Se estiver em outro país, edite o arquivo `chirpstack/configuration/chirpstack/chirpstack.toml` e o packet forwarder para a região correta.

6.2 Cadastro do Dispositivo (Sensor)

1. Device Profiles → Add device profile

- Name: AU915-OTAA
- Region: AU915
- MAC Version: 1.0.3 (ou conforme o sensor)
- OTAA: habilitado

2. Applications → Add application

- Name: Sensores

3. Dentro da aplicação: Add device

- Informe o **Device EUI**, **App Key** e selecione o perfil criado

A partir deste momento, todo **uplink** do sensor será inserido automaticamente na tabela `lorawan_data` do Supabase.

7 Comandos Úteis

Tabela 2: Comandos de operação e monitoramento

Ação	Comando
Ver logs do gateway	<code>docker compose -f packet-forwarder/docker-compose.yml logs -f</code>
Ver logs do ChirpStack	<code>docker compose logs -f chirpstack</code>
Ver logs do bridge	<code>sudo journalctl -u lorawan-bridge -f</code>
Parar todos os serviços	<code>docker compose down</code>
Parar packet forwarder	<code>docker compose -f packet-forwarder/docker-compose.yml down</code>
Descobrir EUI do gateway	<code>docker compose -f packet-forwarder/docker-compose.yml run -rm udp-packet-forwarder find_concentrator</code>
Listar containers ativos	<code>docker ps</code>

8 Solução de Problemas

8.1 Gateway aparece “Offline”

- Verifique se o packet forwarder está em execução: `docker ps`
- Confirme se o EUI cadastrado no ChirpStack é **exatamente igual** ao do `.env`
- Consulte os logs: `docker compose -f packet-forwarder/docker-compose.yml logs`

8.2 Erro sx1261 nos logs do packet forwarder

O RAK7371 **não possui** o rádio SX1261. Desative a referência no `global_conf.json` do container ou ignore o *warning* se o concentrador iniciar normalmente.

8.3 Dados não chegam no Supabase

- Verifique se o bridge está rodando e conectado ao MQTT
- Confirme se a `SUPABASE_KEY` é a `service_role` key
- Verifique se a tabela `lorawan_data` foi criada corretamente no SQL Editor

8.4 Payload chega em base64

Configure um **Codec** no *Device Profile* do ChirpStack (JavaScript ou Cayenne LPP) para decodificar o payload binário em JSON antes do envio ao Supabase.

8.5 Problemas específicos do Armbian

8.5.1 RAK7371 não aparece no `lsusb`

- Tente outra porta USB (preferencialmente USB 3.0)
- Verifique se a placa possui fonte de alimentação de pelo menos 2.5A
- Instale/atualize o pacote `usbutils`: `sudo apt install usbutils`

8.5.2 Erro de permissão USB

Adicione regras `udev`:

```
sudo tee /etc/udev/rules.d/99-rak7371.rules > /dev/null <<EOF
SUBSYSTEM=="usb", ATTR{idVendor}=="10c4",
    ATTR{idProduct}=="ea60", \
    MODE="0666", GROUP="dialout"
EOF
sudo udevadm control --reload-rules
sudo udevadm trigger
```

8.5.3 Containers reiniciam em loop

Provavelmente falta memória. Verifique com:

```
free -h
docker stats
```

Aumente o `swap` (Seção 2.2) ou utilize uma placa com mais RAM.

9 Considerações de Segurança

- O arquivo `.env` está no `.gitignore` e **nunca deve ser commitado**
- A `service_role` key do Supabase concede acesso total ao banco; mantenha-a em ambiente seguro
- Altere a senha padrão do ChirpStack (`admin/admin`) assim que possível
- Considere implementar *Row Level Security* (RLS) no Supabase para ambientes de produção

10 Conclusão

Com este manual, o gateway LoRaWAN está totalmente operacional no Armbian, integrando dados de sensores diretamente no Supabase. A arquitetura containerizada facilita manutenção, backup e replicação do ambiente.

Para atualizações e melhorias no projeto, consulte o repositório oficial:

<https://github.com/WCosmo/lorawan-rak7371-supabase>

Wilson Cosmo
github.com/WCosmo

Este manual foi elaborado para auxiliar na implantação de gateways LoRaWAN em ambientes ARM/Armbian, promovendo a democratização do acesso às redes de baixa potência e longo alcance.